Singly Linked List – Search Element In Sorted List Iterative and Time Complexity.

Program:

```
bool searchElementInSortedListIterative(Node **head, int key)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return false;
    }
    else
    {
        Node *current = *head;
        while (current != nullptr)
        {
            if (current->data == key)
            {
                return true;
            }
            current = current->next;
        }
        return false;
    }
}
….
 case 10:
        {
            int key;
            cout << "Enter the element to search: ";
            cin >> key;
```

```
        if (searchElementInSortedListIterative(&head, key))
        {
            cout << "Element found." << endl;
        }
        else
        {
            cout << "Element not found." << endl;
        }
        break;
    }
```

$\boldsymbol{First\ checking\ is\ whether\ head\ pointer\ value\ is\ null\ i.e.\ List\ is\ empty}$:

$\boldsymbol{Pictorially\ depicted\ as}$:

$\boldsymbol{Node * head}$	
$\boldsymbol{Address}$	$\boldsymbol{Head}$
$ff21$	$\boldsymbol{nullptr}$

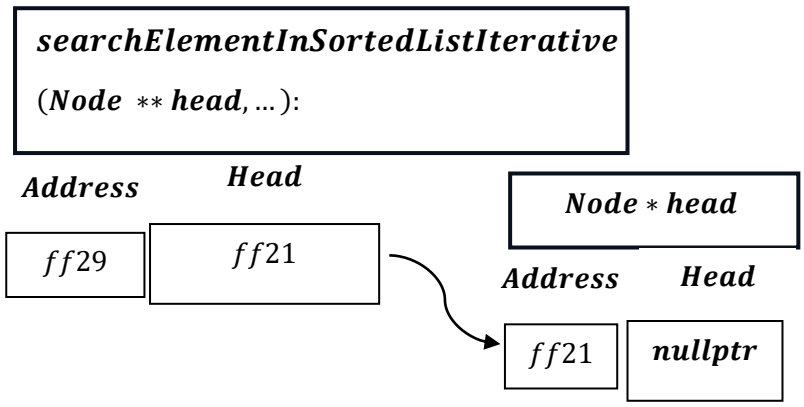
```
        if (searchElementInSortedListIterative(&head, key))
        {
            cout << "Element found." << endl;
        }
        else
        {
            cout << "Element not found." << endl;
        }
```

$\boldsymbol{if\left(searchElementInSortedListIterative(ff21, 3)\right)\{\ldots\}else\{\ldots\};}$

$pointer\ head's\ address\ passed\ is\ ff21\ in\ function\ call\ (pass\ by\ reference)$

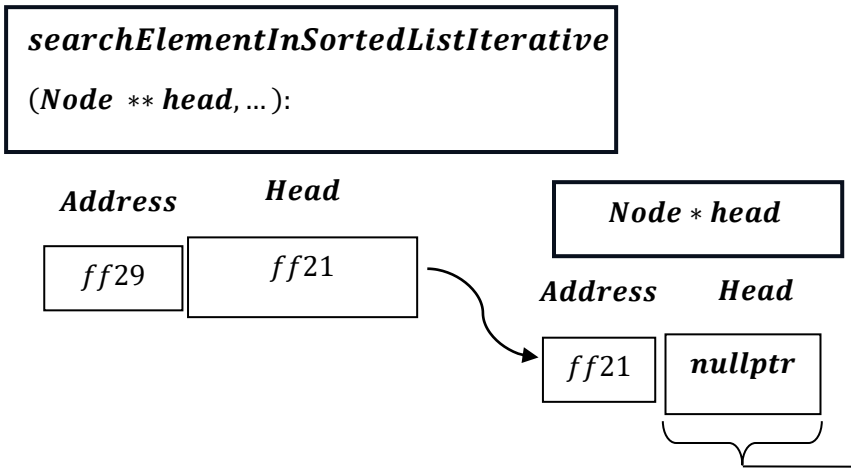
$and\ key\ is : 3$.

```
bool searchElementInSortedListIterative(Node **head, int key){…}
```



$if(*\ head == nullptr)\ if\ it\ is\ true$

$If\ true\ then\ Why\ true\ ?$
$Ans: Though\ *\ head\ which\ is\ of\ Node\ **\ head$
$temporary\ pointer\ to\ pointer\ variable\ of\ the\ function$
$, as\ *\ head\ is\ used\ , it\ dereference\ from\ Node\ **\ head\ and$
$refer\ to\ Node\ *\ head\ global\ pointer\ to\ which\ Node\ **\ head$
$is\ pointing\ to\ and\ then\ its\ value\ is\ checked\ is$
$whether\ equal\ to\ nullptr\ or\ not\ ?\ This\ occurs\ during\ execution$
$and\ compilation\ of\ the\ searchElementInSortedListIterative\ function.$

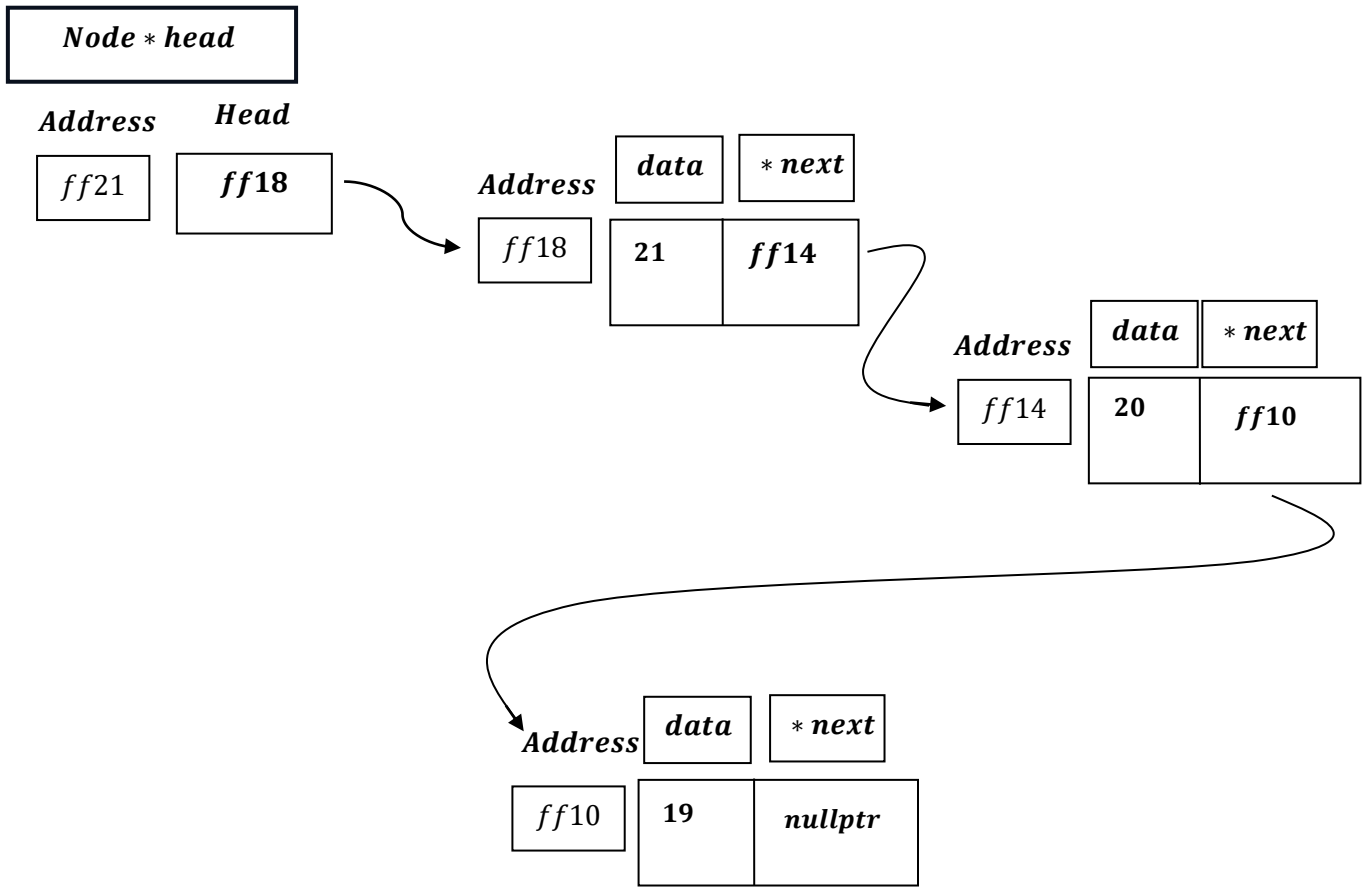


```
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return false;
    }
```

If List is Empty , then , it prints a message : `Error: List is empty.` Here input position entered by user doesn't matters. And returns false.

Now if Linked List is not Empty:

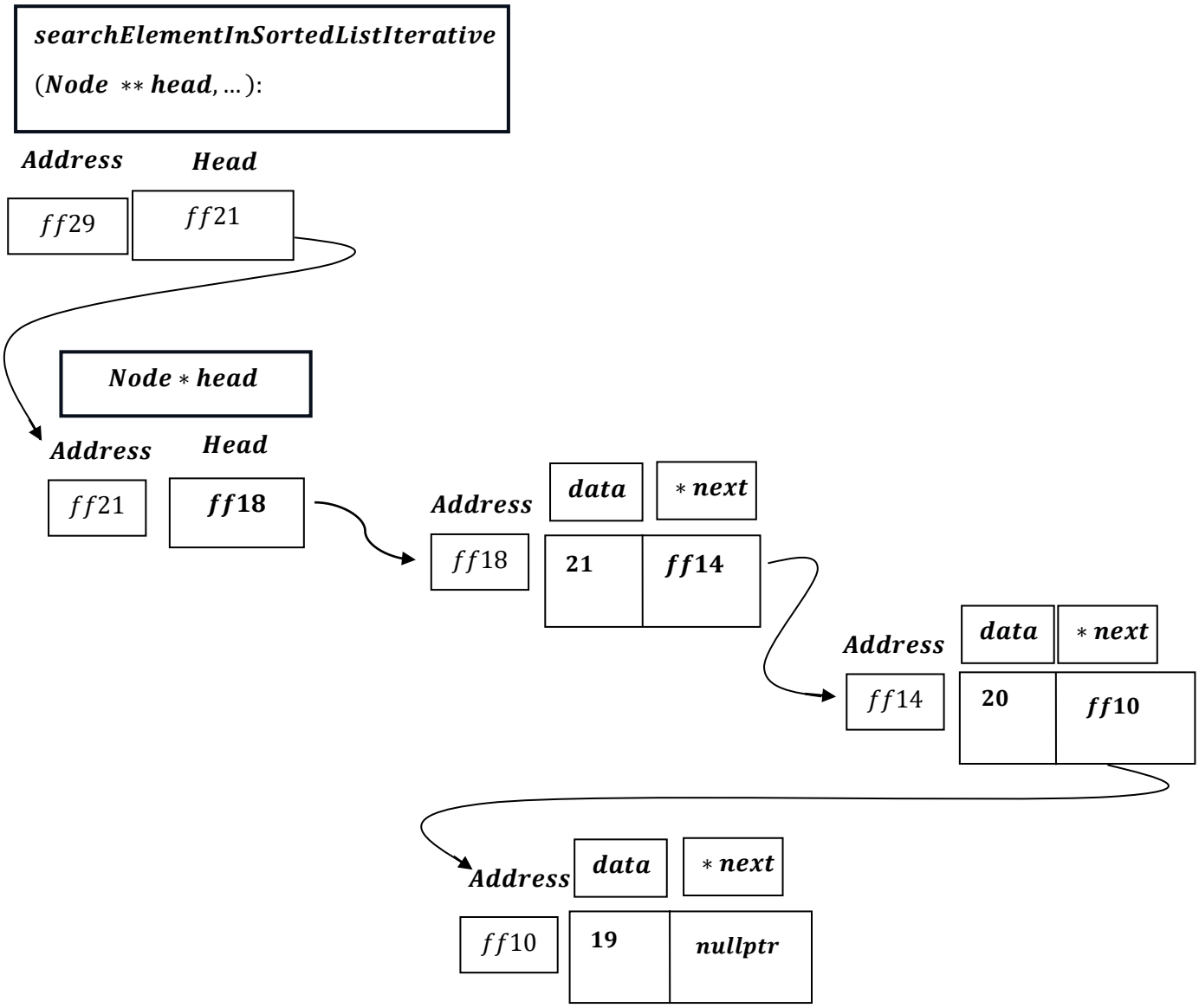
We have linked list :



```
        if (searchElementInSortedListIterative(&head, key))
        {
            cout << "Element found." << endl;
        }
        else
        {
            cout << "Element not found." << endl;
        }
```

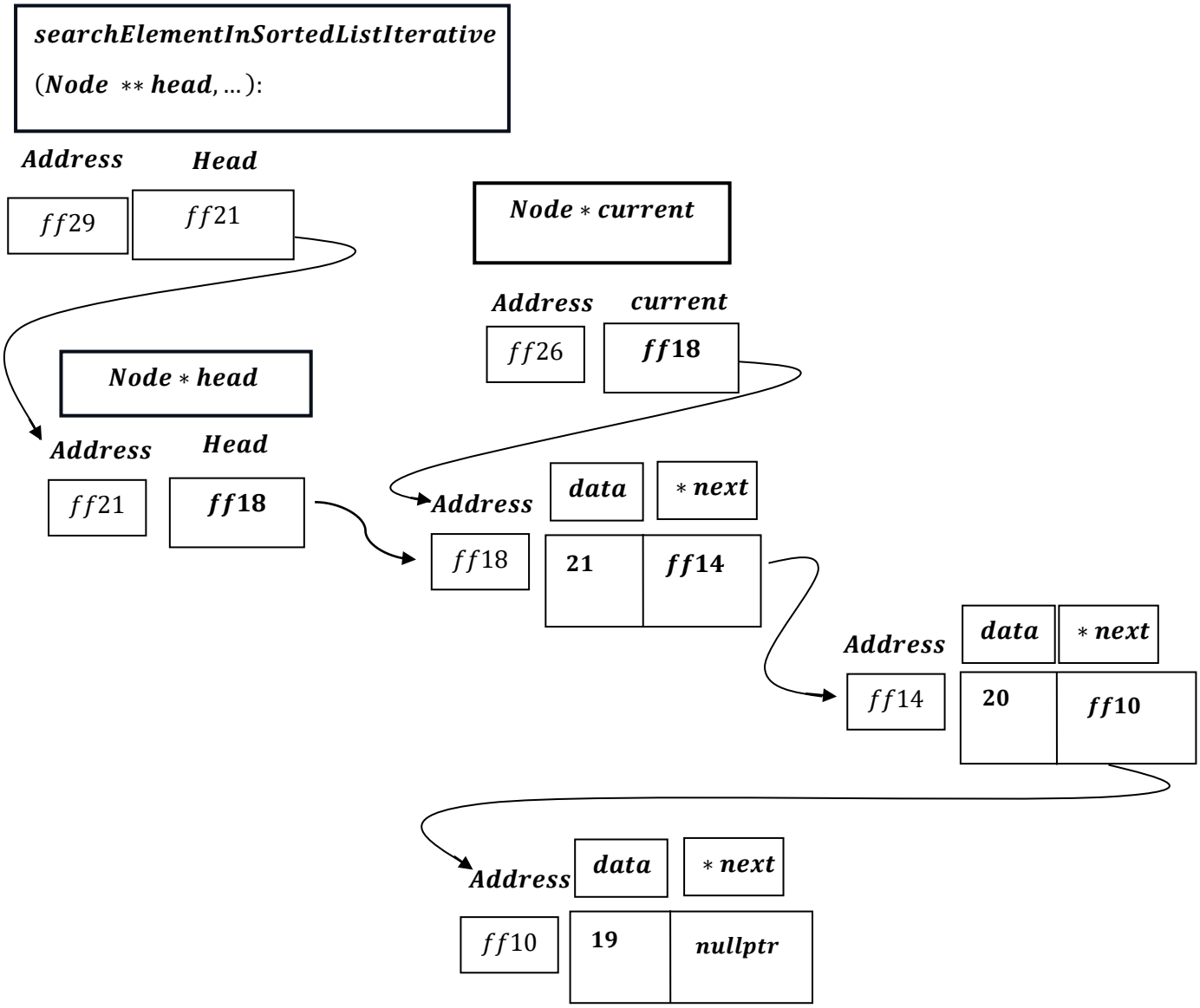
$if(searchElementInSortedListIterative(ff21, 19))\{\ldots\}else\{\ldots\}; i.e., pointer\ head's\ address\ passed\ is\ ff21\ in\ function\ call\ (pass\ by\ reference) and\ key\ is: 19.$

```
bool searchElementInSortedListIterative(Node **head, int key){…}
```



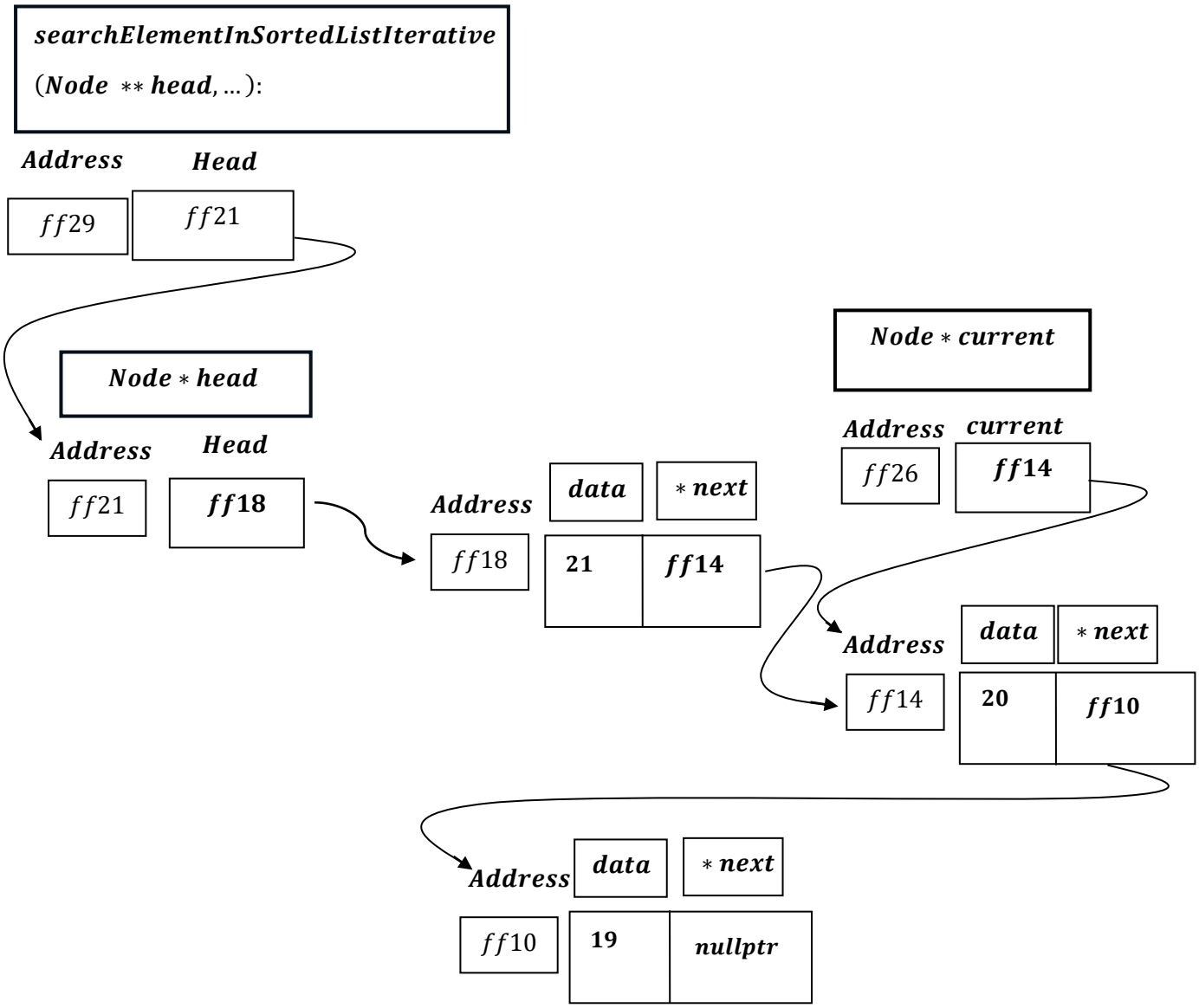
```
else
{
    Node *current = *head;
    while (current != nullptr)
    {
        if (current->data == key)
        {
            return true;
        }
        current = current->next;
    }
    return false;
}
```

```
    Node *current = *head;
```



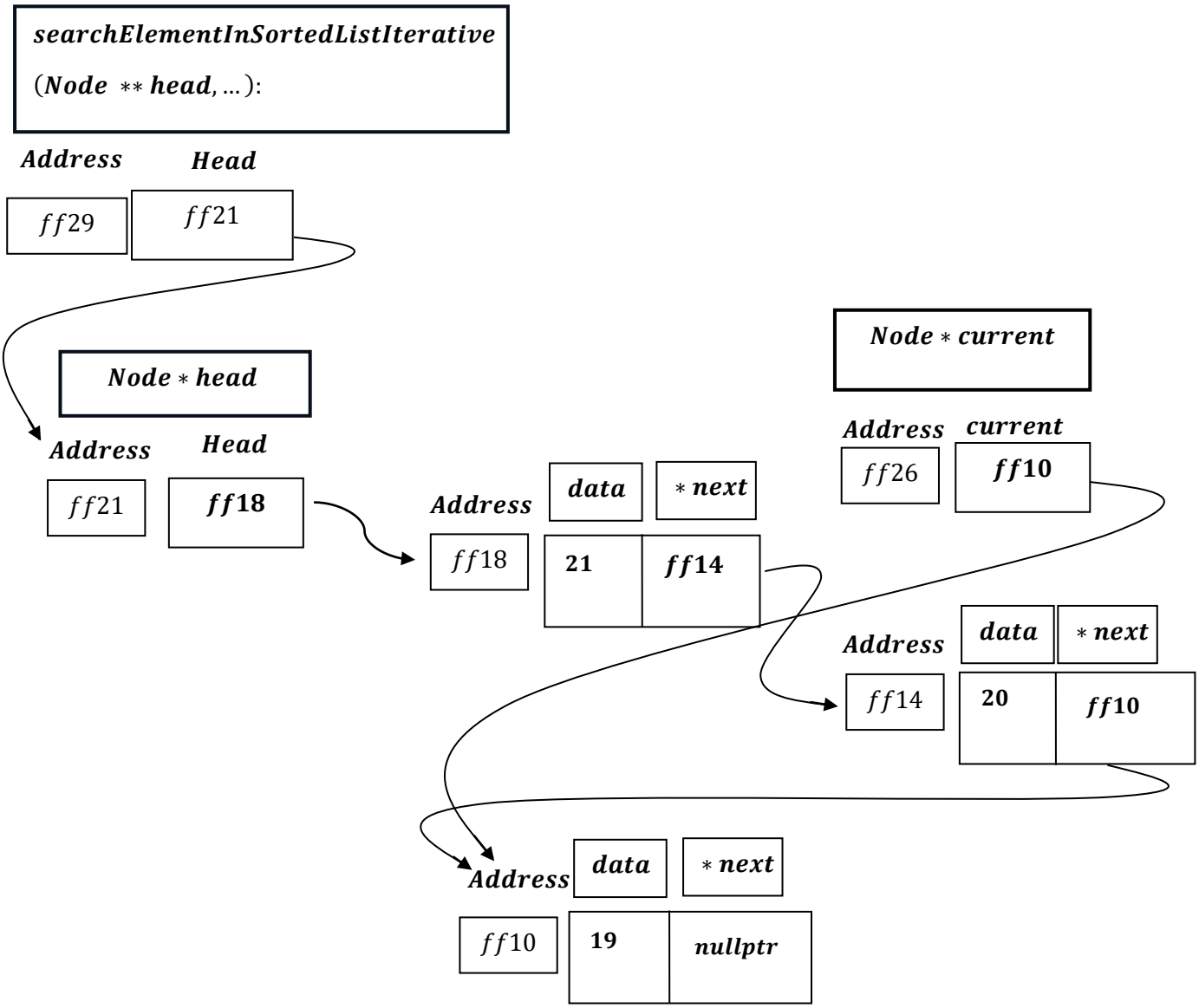
```
while (current != nullptr)
        {
            if (current->data == key)
            {
                return true;
            }
            current = current->next;
        }
```

Now, current's value is ff18 and current → data = 21 and Key is 19 , hence if's condition is not true , therefore, current = current → next = ff14.



$Now, current's\ value\ is\ ff14\ and\ current \rightarrow data = 20\ and\ Key\ is\ 19\ , hence\ if's$

$condition\ is\ not\ true\ , therefore, current = current \rightarrow next = ff10.$



$Now, current's\ value\ is\ ff10\ and\ current \rightarrow data = 19\ and\ Key\ is\ 19\ , hence\ if's$

$condition\ is\ \ true, hence\ return\ true\ and\ loop\ ends\ .$

$Then\ in\ Switch - Case\ (Case\ 10):$

```
        if (searchElementInSortedListIterative(&head, key))
        {
            cout << "Element found." << endl;
        }
```

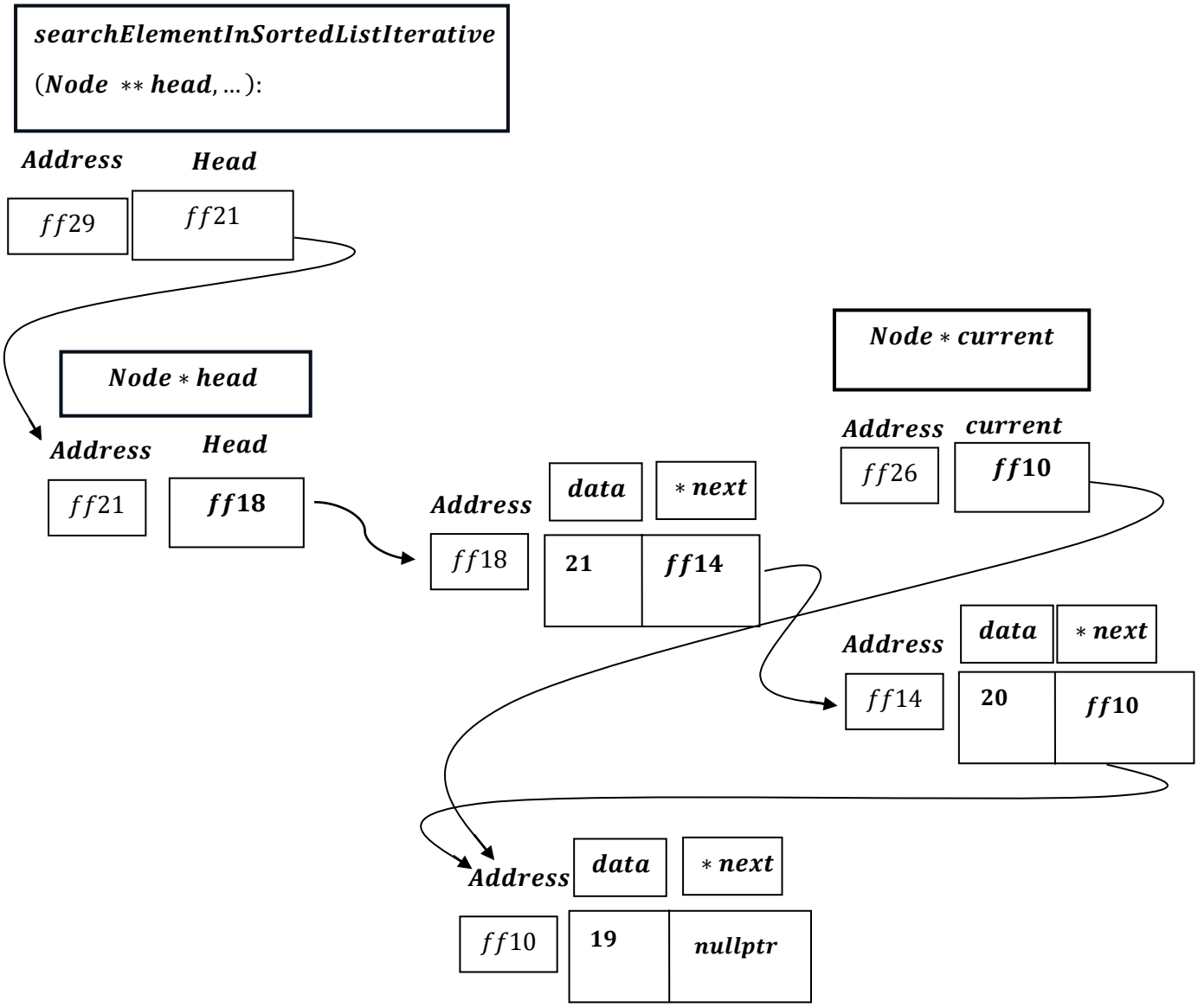
$We\ can\ write\ if\big(searchElementInSortedListIterative(\&head, key)\big)\ or$

$if(searchElementInSortedListIterative(\&head, key) == true)$, $both\ are\ same$.

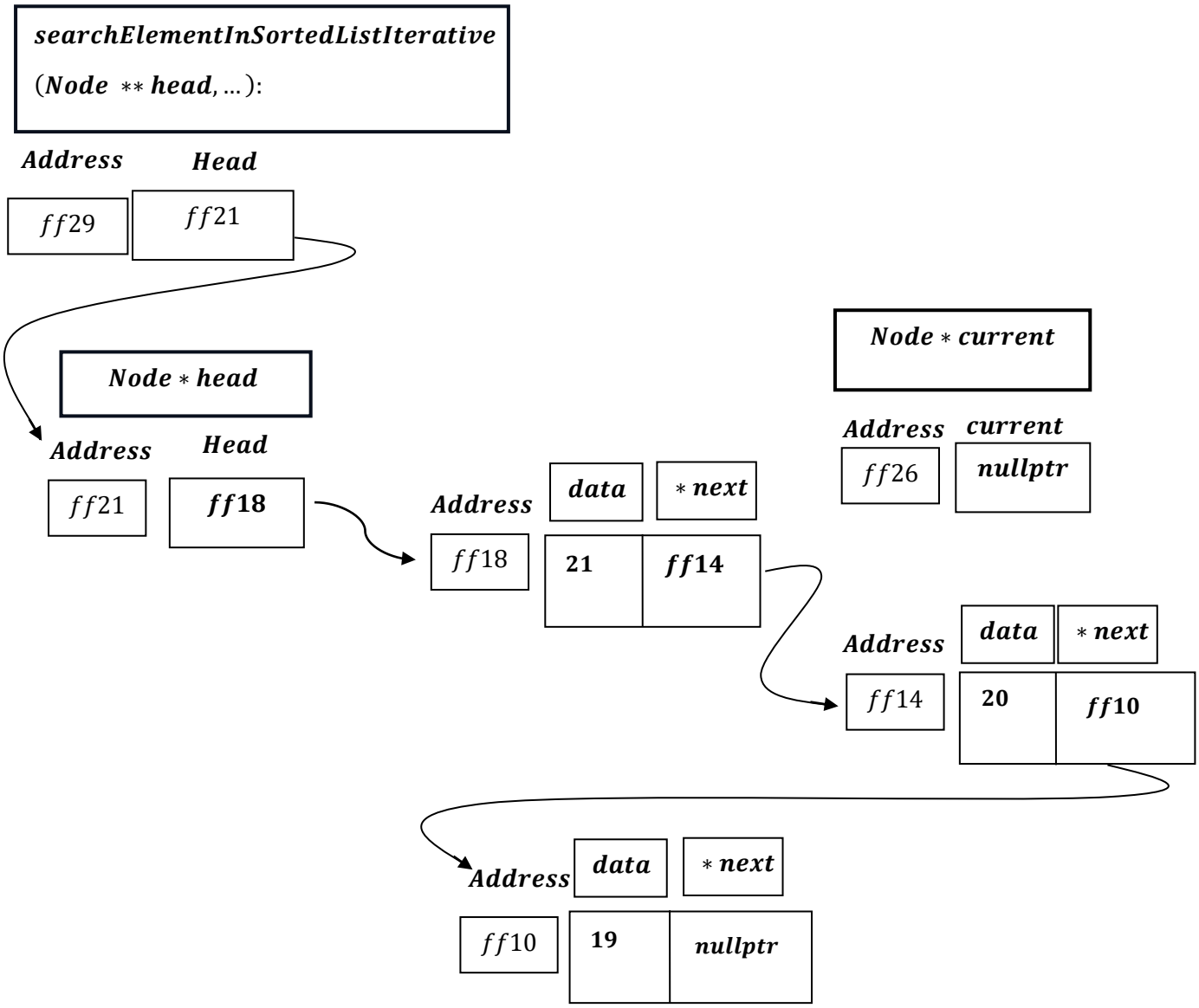
$Hence\ as, if\ condition\ is\ true$, $it\ prints$ "$Element\ is\ found$" .

$Now, if\ key\ is\ not\ found$

$Say\ key\ is\ 18\ and\ \ not\ in\ the\ list. And\ we\ in\ the\ stated\ below\ condition$:



$Hence, current's\ value\ is\ ff10\ and\ current \rightarrow data = 19\ and\ Key\ is\ 18\ , hence\ if's\ condition\ is\ not\ true\ , therefore, current = current \rightarrow next = nullptr.$



$Now\ as\ soon\ as\ current\ value\ gets\ updated\ to\ `nullptr`\ loop\ ends\ and$

```
    else
    {
        Node *current = *head;
        while (current != nullptr)
        {
            if (current->data == key)
            {
                return true;
            }
            current = current->next;
        }
        return false;
    }
```

$function\ returns\ false.$

$As\ soon\ as\ ,function\ returns\ false\ ,we\ come\ back\ to\ the\ switch-case:$

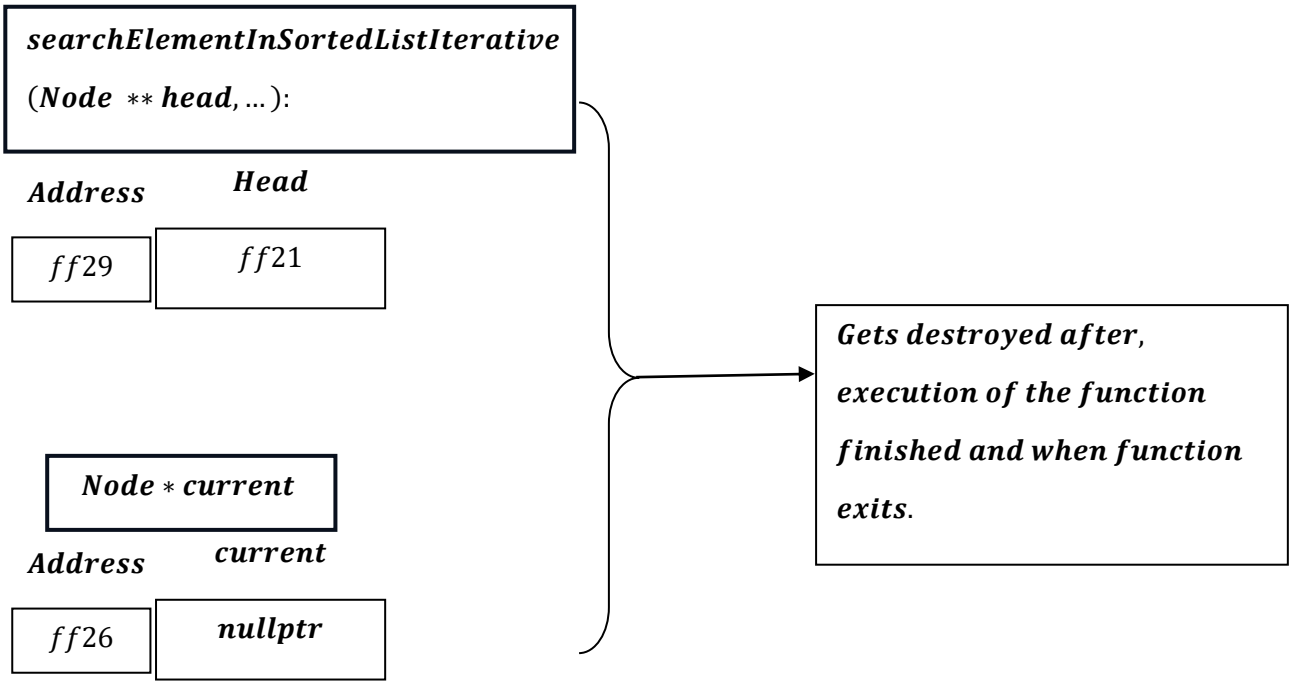
```
            if (searchElementInSortedListIterative(&head, key)){….}
            else
            {
                cout << "Element not found." << endl;
            }
```

$if\big(searchElementInSortedListIterative(\&head, key)\big)\ is\ false\ \ or$

$if(searchElementInSortedListIterative(\&head, key) == true)\ is\ false$

$,then\ it\ moves\ to\ the\ else\ part\ and\ it\ prints\ "Element\ not\ found"\ .$

After the function module ends , all temporary pointer variables created gets destroyed, i.e.



The searchElementInSortedListIterative function step by step explanation:

Function Definition:

→ ***Return Type***: `***bool***` − ***The function returns a boolean value*** (`***true***` ***or*** `***false***`).

→ ***Parameters***:

- `***Node ** head***`: ***A pointer to a pointer to the head node of the linked list.***
- `***int key***`: ***The value to search for in the linked list.***

1. *Handling an Empty List*:

– *The function first checks if the linked list is empty by examining the `*head` pointer.*

– *If `*head` is `nullptr`, it means the list is empty.*

– *An error message "Error: List is empty." is printed to the console.*

– *The function returns `false` to indicate that the key was not found because the list is empty.*

2. *Traversing the List*:

– *If the list is not empty, the function proceeds to traverse the list.*

– *A pointer named `current` is initialized to `*head`. This pointer is used to traverse through the linked list.*

3. *Comparing with the Key*:

– *Inside the `while` loop, the `data` member of the `current` node is compared with the `key` being searched.*

– *If a match is found (`current → data == key`), the function immediately returns `true`, indicating that the key is present in the list.*

4. *Moving to the Next Node*:

– *If the current node's `data` doesn't match the `key`, the `current` pointer is advanced to the next node in the list using `current = current–> next`.*

– *This process of comparison and advancement continues until either the key is found or the end of the list is reached.*

5. *Key Not Found*:

– *If the loop completes without finding the `key` (i.e., it reaches the end of the list), the function returns `false` to signal that the `key` was not present in the linked list.*

Time Complexity of searchElementInSortedListIterative Function:

```
bool searchElementInSortedListIterative(Node **head, int key)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return false;
    }
    else
    {
        Node *current = *head;
        while (current != nullptr)
        {
            if (current->data == key)
            {
                return true;
            }
            current = current->next;
        }
        return false;
    }
}
```

$O(1)$

$O(1)$

runs `N` times in worst time complexity.

runs `N` times in worst time complexity.

runs N or `N − 1` times in worst time complexity.

$O(1)$

There are three scenarios:

1. *Best Case*: *When List is empty or Loop runs only once i.e. Key found at head node of the list.*

2. *Worst Case*: *When Key remain not found or found at Tail Node or Last Node.*

3. *Average Case*: *If the elements in the linked list are uniformly distributed and the key is equally likely to be any element in the list.*

Best Case

1)*When List is empty*:

– The function first checks if the linked list is empty by examining the `∗ head` pointer.

– If `∗ head` is `nullptr`, it means the list is empty.

– An error message "Error: List is empty." is printed to the console.

– The function returns `false` to indicate that the key was not found because the list is empty.

This whole process takes constant amount of time i.e. $\Omega(1)$.

2)*When Key present at Head node*:

–Now head pointer doesnot carries `Null Pointer ` value, hence it checks at if condition first , such checking takes constant amount of time i.e. $O(1)$.

–Next it moves to else part , and current pointer gets assigned to Head pointer's value which is head node's address and it takes constant amount of time i.e. $O(1)$.

–Next it enters in the loop , and as current is not null pointer , hence if checks that current pointer points to the `data` contained memory location pointed by the current pointer equals to searched Key and as it is the best case hence it will match and function returns true . As, loop runs only once , hence it takes constant amount of time i.e. $O(1)$.

$$\textit{Hence overall time complexity}:$$

$$O(1) + O(1) + O(1) = O(1).$$

As we measure best time complexity on Big − Omega `Ω` , hence overall time complexity : $\Omega(1)$.

Worst Case

1) *When Key found at Last Node :*

− Head pointer doesnot carries `Null Pointer` value, hence it checks at if condition first , such checking takes constant amount of time i.e. $O(1)$.

−Next it moves to else part , and current pointer gets assigned to Head pointer's value which is head node's address and it takes constant amount of time i.e. $O(1)$.

−Next it enters in the loop , and as current is not null pointer , hence if checks that current pointer points to the `data` contained memory location pointed by the current pointer equals to searched Key:

→ *When Current pointer is pointing at Head Node, Loop runs* 1 *time* :

→ *If condition checks* 1 *time* .

→ *Current = Current → Next runs* 1 *time*.

→ *When Current pointer is pointing at* 2nd *Node, Loop runs* 1 *time* :

→ *If condition checks* 1 *time* .

→ *Current = Current → Next runs* 1 *time*.

.

.

$\rightarrow$ $When\ Current\ pointer\ is\ pointing\ at\ N-1^{th}\ Node, Loop\ runs\ 1\ time$:

$\rightarrow$ $If\ condition\ checks\ 1\ time$.

$\rightarrow$ $Current = Current \rightarrow Next\ runs\ 1\ time.$

$\rightarrow$ $When\ Current\ pointer\ is\ pointing\ at\ N^{th}\ Node, Loop\ runs\ 1\ time$:

$\rightarrow$ $If\ condition\ checks\ 1\ time$.

$\rightarrow$ $Current = Current \rightarrow Next\ runs\ 0\ time, as\ this\ statement\ didn't$ $get\ executed\ due\ to\ if\ condition\ becomes\ true\ and\ it\ returns\ true.$

$Hence\ checking\ of\ if\ Condition\ is\ runs\ : N\ times$,

$While$, $Current = Current \rightarrow Next\ runs : N-1\ times.$

$Therefore\ total\ time\ complexity$: $N + N - 1 = 2N - 1.$

$Applying\ Big-O\ \ for\ the\ worst\ time\ complexity$: $O(2N-1) = O(N).$

$Hence\ total\ time\ complexity\ of\ the\ whole\ process$:

$$O(1) + O(1) + O(N) = O(N).$$

2) *When Key is not found* :

1) *When Key found at Last Node* :

$-$ *Head pointer doesnot carries `Null Pointer` value, hence it checks at if condition first , such checking takes constant amount of time i.e.* $O(1)$.

$-$*Next it moves to else part , and current pointer gets assigned to Head pointer's value which is head node's address and it takes constant amount of time i.e.* $O(1)$.

$-$*Next it enters in the loop , and as current is not null pointer , hence if checks that current pointer points to the `data` contained memory location pointed by the current pointer equals to searched Key*:

$\rightarrow$ *When Current pointer is pointing at Head Node, Loop runs* 1 *time* :

$\rightarrow$ *If condition checks* 1 *time* .

$\rightarrow$ *Current* $=$ *Current* $\rightarrow$ *Next runs* 1 *time*.

$\rightarrow$ *When Current pointer is pointing at* 2nd *Node, Loop runs* 1 *time* :

$\rightarrow$ *If condition checks* 1 *time* .

$\rightarrow$ *Current* $=$ *Current* $\rightarrow$ *Next runs* 1 *time*.

.

.

$\rightarrow$ *When Current pointer is pointing at* $N-1^{th}$ *Node, Loop runs* 1 *time* :

$\rightarrow$ *If condition checks* 1 *time* .

$\rightarrow$ *Current* $=$ *Current* $\rightarrow$ *Next runs* 1 *time*.

$\rightarrow$ *When Current pointer is pointing at* N^{th} *Node, Loop runs* 1 *time* :

$\rightarrow$ *If condition checks* 1 *time* .

$\rightarrow$ *Current* $=$ *Current* $\rightarrow$ *Next runs* 1 *time. As current gets assigned to `NullPtr` and exits from while loop.*

Hence checking of if Condition is runs : N *times* ,

While , *Current* $=$ *Current* $\rightarrow$ *Next runs* : N *times.*

Therefore total time complexity: $N + N = 2N$.

Applying Big $-$ O *for the worst time complexity*: $O(2N) = O(N)$.

$-$*As function returns false, it takes constant amount of time i.e.* $O(1)$.

Hence total time complexity of the whole process:

$$O(1) + O(1) + O(N) + O(1) = O(N).$$

Average Case

(*If the elements in the linked list are uniformly distributed and the key is equally likely to be any element in the list*)

$-$*if key is found at* 1st *Node* , *loop runs* 1 *time* .

$-$*if key is found at* 2nd *Node* , *loop runs* 2 *time* .

.

.

$-$*if key is found at* N^{th} *Node, loop runs* N *times.*

$As\ , the\ key\ is\ equally\ likely\ to\ be\ any\ element\ in\ the\ list,$

$probablity\ of\ finding\ 1^{st}\ element\ (p_1), if\ key\ equals\ to\ 1^{st}\ element\ = \frac{1}{N}.$

$probablity\ of\ finding\ 2^{nd}\ element\ (p_2), if\ key\ equals\ to\ 2^{nd}\ element\ = \frac{1}{N}.$

.

.

$probablity\ of\ finding\ N^{th}\ element\ (p_2), if\ key\ equals\ to\ N^{th}\ element\ = \frac{1}{N}.$

$Therefore,$

$$Probability(P) = p_1 = p_2 = \cdots = p_n = \frac{1}{N}.$$

$Therefore, Average\ Case = Number\ of\ times\ the\ loops\ executes\ \times$
$probablity\ of\ finding\ elements.$

$That\ is\ the\ average\ number\ of\ executions\ is\ the\ sum\ of\ all\ possible\ executions$
$weighted\ by\ their\ probabilities. This\ is\ given\ by:$

$$\therefore Average\ Case = \frac{1}{N} \times \left(\sum_{i=1}^{N} i\right)$$

$$= \frac{1}{N} \times \frac{N(N+1)}{2} \left[\sum_{i=1}^{N} i = \frac{N(N+1)}{2}\right]$$

$$= \frac{N+1}{2}$$

$$= \frac{N}{2} + \frac{1}{2}$$

$\boldsymbol{Applying\ , Big\ Theta\ \Theta\ we\ get}$:

$$= \frac{1}{2} \times \left(\Theta(N) + \Theta(1)\right)$$

$$= \Theta(N) + \Theta(1)$$

$$= \Theta(N).$$

$\boldsymbol{Note\ , by\ relation}: \boldsymbol{\Omega \leq \Theta \leq O\ , we\ get}$:

$$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Longrightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$$

Singly Linked List	***Cases***	***Time Complexity***
Search Element In SortedList Iterative	**1.** ***Best Case***	$\Omega(1)$
	2. ***Worst Case***	$O(n)$
	3. ***Average Case***	$\Theta(n)$

****